Lab 4: Data Mining
ISC 4221
October 11, 2021

Programming Language: Matlab or Python
Due date: 11:59PM on November 1, 2021
Submit online through Canvas
To receive full credit, you must submit:

1. Well-commented and working code
2. A single PDF writeup containing any plots, discussion, calculations, and conclusions.

Goals: The objectives of this lab are to use hierarchical clustering to analyze a data set and to use k-means clustering to compress a color image.

---

1. Hierarchical clustering of 2020 election data The goal of clustering is to group objects together with similar characteristics. In this problem, we will cluster the 50 U.S. states in terms of their political makeup after the 2020 election. The data is contained in the file `states.txt`. Each row of this file lists, from left to right,

- State name
- Percentage of the Republican senators;
- Percentage of the Republican congressmen;
- Percentage of the Republican governors;
- Percentage of the Republican electoral college.

Note that the last two entries must either be 0 or 1, and the second entry must be either 0, 0.5, or 1 (each state has two senators).

(i) Load the file `states.txt` and separate the state names from the percentages.

(ii) Using the Euclidean distance, compute the dendrogram tree using
   - single linkage,
   - complete linkage,
   - centroid linkage,
   - average linkage.

   Orient your tree so that the increasing cluster sizes go from right to left rather than bottom to top. Label the y-axis with the state names. You may use built-in Matlab or Python routines to compute these dendrogram trees.

(iii) Comment on any common characteristics between the states.

---

2. Image Compression with k-means In a color image, each pixel is represented by three RGB values creating a myriad of colors. Our goal here is to choose just a few colors to represent the image.[1] After we choose these colors, then the image chart for the picture

[1] An obvious application of this data compression is when you print an image using a color printer with many fewer colors than are available on your computer.

must be modified so that each color is replaced by the new color that it is closest in color space. In Matlab, the routines `imread`, `imwrite`, and `imshow` can be used. Similar routines exist in Python.

We can use k-means to accomplish this image compression. For example, suppose we want to represent an image with 32 colors. Our job is to find which 32 colors (out of 2^{24} possible) that best represent the image. We then initiate our probabilistic Lloyds algorithm with 32 generators which are 3 dimensional integers ex. (25, 251, 100) corresponding to RGB shades. For this application, use the standard Euclidean distance as the distance metric.

After convergence is achieved with k-means we know the best 32 colors to represent our image. Our final step is to replace each color in our original matrix representation of the image with the converged centroid of the cluster it is in.

Figure 1: The image `mandrill.tiff`

(i) Obtain approximations to the image using $k = 2$, $k = 4$, and $k = 8$ colors. For a stopping criteria, use 10 steps of Lloyd's algorithm. Compute and plot the energy versus the iteration number on a linear-linear graph. Comment on its behavior.

(ii) For each k, display your compressed image along with the original image.